One highlight, from capture to report

Demo walkthrough without Vergil

Current commands and reviewer controls | Sanitized copy

Stage, inspect, apply and report using the three-row SharePoint demo. A separate eight-minute GitLab rehearsal also demonstrates optional graduation. All entries are TEST ONLY; screenshots preserve dated cloud evidence.

01 // Stage

Capture exact receipt rows with required correction ancestry

02 // Inspect

Read the proposed changes and verify the selected destination

03 // Apply

Confirm the reviewed entries with the Yes / No buttons

04 // Report

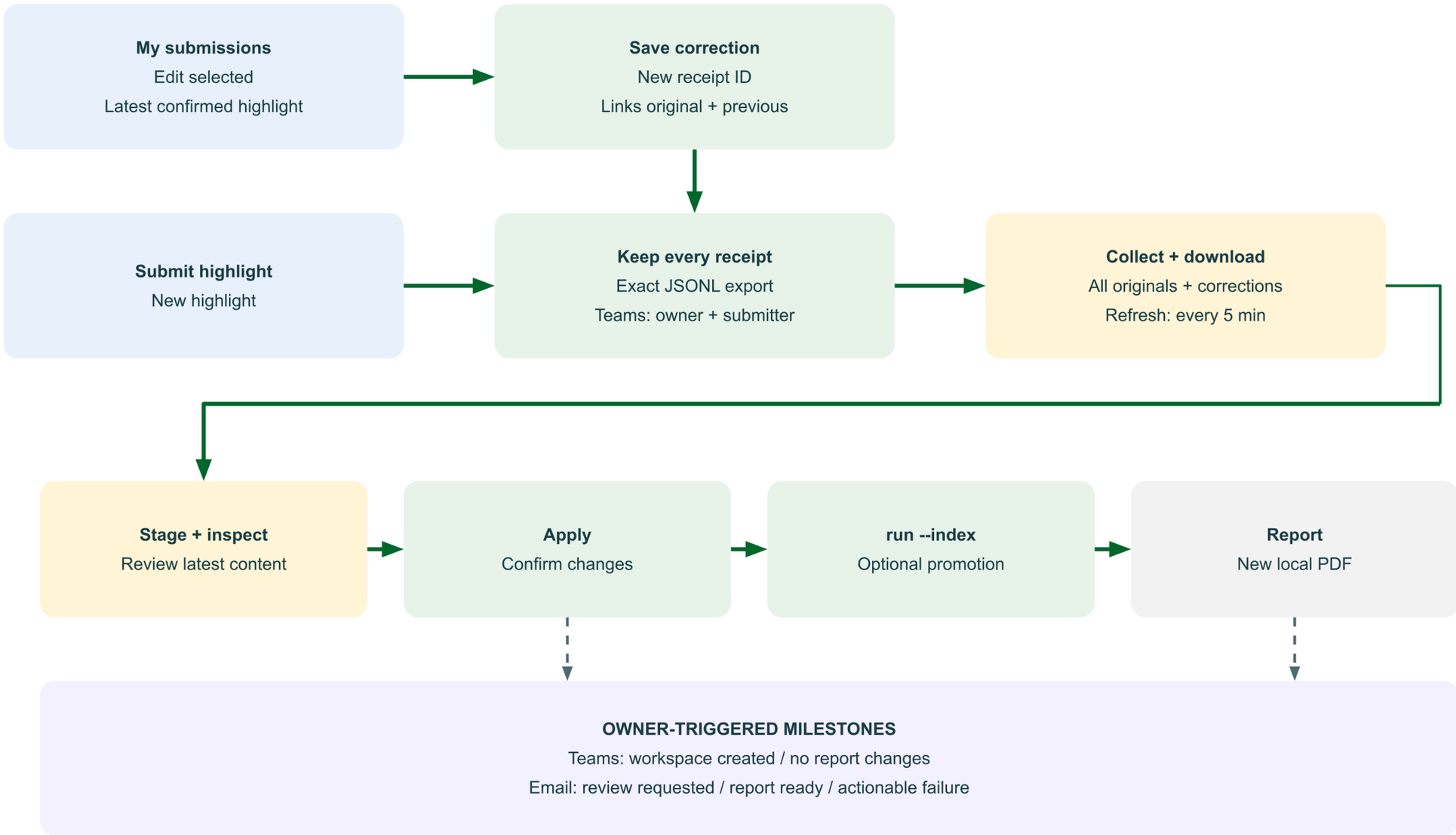
Generate one combined local PDF; graduation is optional

The following map shows the active SharePoint capture route. New-save Teams notices are automatic and best effort. Later review/report messages use the owner-triggered callback. The retained full rehearsal uses the GitLab fixture.

WORKFLOW MAP

From highlight to reviewed report

New highlights and later corrections follow the same SharePoint review and reporting path.



Solid green: saved data and review steps. A newly exported original or correction triggers Teams, best effort. Dashed: owner-triggered milestones. Corrections do not alter existing reports; local CLI commands send no messages.

Demo scope and evidence

Updated October 27, 2026. **SharePoint is the active default; GitLab intake and catalog flows are Off.** The contributor/reviewer and developer guides cover the published form, immutable corrections and notifications. Historical screenshots and acceptance records retain their dates; they are not fresh cloud runs.

Start here

- Presentation walkthrough
- User and reviewer guide
- Developer guide
- SharePoint demo report
- CLI guide

The manual MVP is accepted: browser download → stage → inspect → confirmed apply → combined local PDF. Standard-license contributor access, recovery, collected-export boundaries, native restore, deployed exporter repair and contributor editing are accepted. Wider shared rollout remains PA-12; standalone reporting for the separate `general` destination remains PA-29. Direct CLI OAuth download and Vergil are separate unfinished plans.

Eight-minute SharePoint presentation

Time	Action	Point to show
0:00–1:30	Open the SharePoint form and keep Project highlight selected. Load projects and submit a clearly labeled TEST ONLY highlight.	Placement precedes Project. A report-wide update needs no project, but its standalone CLI reporting is still pending. Use project highlights for the complete current demo.
1:30–2:30	Read the receipt and Teams confirmation.	JSONL ready verifies the individual receipt file. Teams is best effort; there is no routine submission email.
2:30–3:30	Use My Submissions between the banner and form, then Edit selected and Save correction.	Only the caller's latest receipts are offered. The correction has a new UUID and retains original/previous links; project and placement stay fixed.
3:30–4:30	Check the next collected export and download logbook.jsonl.	The exporter runs every five minutes. Confirm freshness and include required correction ancestry; do not export list CSV or recreate JSON.
4:30–6:30	Stage, inspect and approve apply in a separate demo workspace.	Review the latest logical content. Receipt history stays immutable; repeat apply does not duplicate it.
6:30–7:30	Generate and open the combined local PDF.	Local reporting sends no notifications and does not publish. Metadata graduation is optional.
7:30–8:00	Explain remaining rollout work.	General reporting, shared deployment, direct OAuth download and Vergil are independent follow-ups.

The eight minutes is a presentation allocation. The recorded three-row demo below used original project receipts; correction acceptance is recorded separately in the editing evidence. Use only approved synthetic data and a disposable local destination for a new demonstration.

Capture through SharePoint

Open the SharePoint form. New highlights default to Project highlight. Choose a project after loading the maintained catalog of 13 projects. The reserved general destination is separate and hidden from project choices.

Each original or correction creates one immutable list receipt and one verified UUID-named JSONL file. The five-minute exporter collects all receipts into `logbook.jsonl`. Download that combined file for local intake. A save confirmation does not prove the combined file has refreshed yet. For correction batches, include the predecessor chain in the batch or use previously applied local history; missing ancestry stops review/apply.

```
acorn intake stage --batch --config intake.json --source /path/to/logbook.jsonl
acorn intake inspect --config intake.json --batch-id <batch-ID>
acorn intake apply --config intake.json --batch-id <batch-ID>
```

Direct SharePoint HTTPS capture is not exposed by the CLI yet. Preserve downloaded bytes and IDs. General receipts must remain available for PA-29; do not silently drop them from a mixed batch or create a fake project index. If selecting project-only demo rows, retain the complete download separately and document the exact selection.

SharePoint download to a combined local PDF

The October 26 demonstration used three exact rows selected from the six-row downloaded logbook. Keep their original bytes and UUIDs in `selected-demo.jsonl`; keep the full download as evidence. `intake.json` routes only to the separate demo index, and `report.json` selects that same one-project index. Use the existing prepared workspace, with `acorn` on your PATH:

```
cd output/pdf/sharepoint-demo-20261026
```

1. Stage the three selected rows.

```
acorn --offline intake stage --batch --config intake.json --source selected-demo.jsonl
```

2. Inspect the proposed changes. Replace `<batch-ID>` with the ID printed by stage; use that same ID for apply. The expected result is three accepted rows and no rejected rows.

```
acorn --offline intake inspect --config intake.json --batch-id <batch-ID>
```

3. Apply the reviewed entries to the local demo index. Confirm with the normal Yes / No buttons. Add `--yes` only for an already reviewed unattended run.

```
acorn --offline intake apply --config intake.json --batch-id <batch-ID>
```

4. Generate the combined local PDF.

```
acorn --offline report --config report.json --from 2026-10-26T00:00:00Z --through \
  2026-10-27T00:00:00Z --dry-run --combine
```

Open `.acorn/reports/report.pdf` inside the demo directory. The default root is the current directory; the default selection includes every folder in `report.json` (only `acorn` here); the default output is `.acorn/reports`. Combined mode does not require `--commit`. Keep the explicit window to reproduce the three-row demo; `--through` has no time default, and omitting `--from` includes older history. The shorter commands change the output location from the historical `report/` directory, not the intended content.

PDF rendering uses cached or discoverable Chrome/Chromium by default. If discovery fails, set `CHROME_PATH` to your installed browser before running; offline mode cannot download it. Intake currently requires explicit `--config`, `--source` and batch selectors; those have no defaults to remove. These commands do not publish, send callbacks/messages, drain the SharePoint list, or graduate entries. Graduation is optional work after ingestion, not a prerequisite for this report.

The original evidence and report retain the actual batch ID, explicit historical options and hashes. This defaults-based walkthrough describes equivalent steps; it does not claim a new live submission or acceptance run. The workspace's prepared configs contain local paths: use a separate approved demo destination if recreating it elsewhere, and take a new batch ID from stage.

Where notifications occur

A newly saved and verified SharePoint original or correction sends best-effort Teams confirmation to the owner and submitter; a self-submission skips the duplicate notice. History reads and exact replays send no notice. If export first succeeds only on a later retry, the initial notice is not resent. No routine submission email is sent.

Later milestones use the separate owner-triggered receiver: Teams for workspace created or no report changes; email for review requested, report ready or actionable failure. Local CLI commands do not invoke this receiver. Corrections do not change an existing PDF; generate and review a new report. The page 2 map shows the active route and this manual milestone boundary. See notification routing.

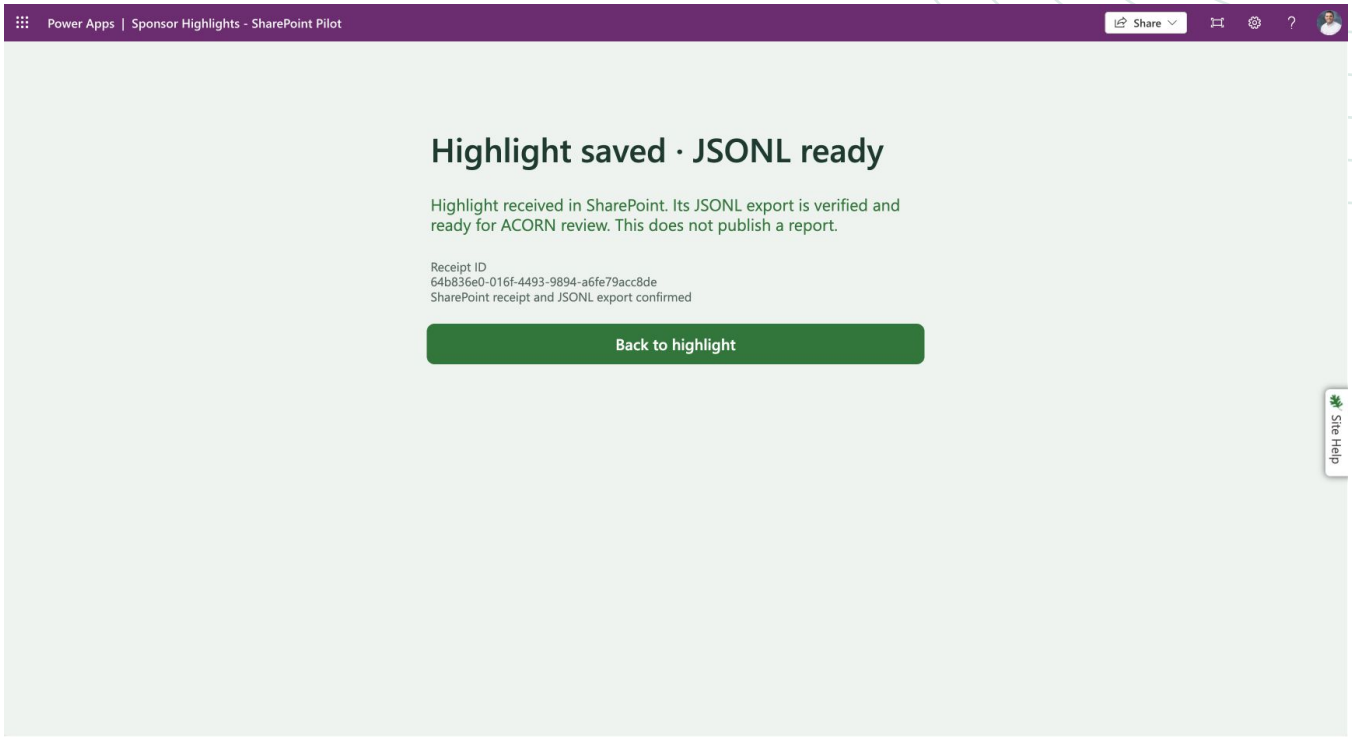
Day-of fallback and boundaries

If approved cloud access fails, use the retained synthetic snapshot and clearly say **recorded cloud capture, live local review and reporting**. Do not enable the paused GitLab flows as an automatic fallback or rotate credentials to evade access controls. Keep unresolved receipts in their original app and browser slot.

Sanitized fixture bytes and native packages may differ from historical hashes. Follow privacy and reuse limits and deliberately rebaseline in an isolated synthetic workspace before rerunning a frozen rehearsal. A historical success record is not verification of a redacted file. Local review/reporting never drains SharePoint or acknowledges rows to a future bot.

SharePoint: verified receipt and export

October 26, 2026: owner-context published app created a list receipt and verified its JSONL export. The Teams action succeeded. The separate three-row owner-context download, stage, inspect, apply and report demonstration also passed October 26. Standard-license participant access, exporter boundary/repair acceptance, native restore and published correction acceptance passed October 27. This screenshot remains a dated original-save example.



Receipt 64b836e0-016f-4493-9894-a6fe79acc8de. One file per receipt; no GitLab append is performed by this branch.

Appendix A — Retained GitLab rehearsal

The original GitLab app and connector are retained, with intake and catalog flows Off. The owner must explicitly restore the route and verify premium rights and connectivity before a new submission. Existing captured files can still be reviewed offline. The following October 24 rehearsal additionally exercises optional contributor graduation; it is historical evidence, not the active capture route.

Appendix A — Repeat safely

Build the current CLI (`cargo build --bin acorn`, default PDF feature), then run from the repository root:

```
python3 .wiki/features/manual-jsonl-ingestion-graduation/_assets/rehearse.py --workspace \
  /tmp/acorn-demo-next-week-01 --acorn target/debug/acorn --chrome \
  '/Applications/Google Chrome.app/Contents/MacOS/Google Chrome'
```

The workspace must not exist. The script refuses to reset an existing directory. Use a new name for each rehearsal; retain earlier evidence. It uses ACORN's **Yes / No** buttons for intake approval, selection of the contributor promotion, and final graduation approval. No is selected by default; Tab or the arrow keys switches selection and Enter confirms it. `y` and `n` also work; Escape or Ctrl+C cancels. No skips a graduation candidate; in this one-candidate rehearsal, skipping it stops the expected graduation checks. Declining the intake or final graduation approval stops the rehearsal without applying that change. Run in a terminal with both input and output attached. Interactive receipts appear in the terminal; the evidence records each command, confirmation mode and exit status.

`--approve-synthetic` permits an unattended check of **only the frozen TEST ONLY fixture**. It supplies `--yes` for intake apply and explicit `--accept` plus the preview digest for graduation. This is not Jason's personal review of production data. Apply replay and rejection checks also pass `--yes` to test validation without additional prompts. `--git /absolute/path/to/git` can select a Git executable if the system toolchain is unavailable.

The current graduation API is:

```
acorn intake run --index project/index.json --dry-run
acorn intake run --index project/index.json --interactive
# Accept all eligible promotions without prompting:
acorn intake run --index project/index.json --force
```

`--force` preserves validation, stale-digest checks and exact backups. Combining it with `--dry-run` writes nothing. The older `candidates` and `graduate` commands visible in historical transcripts still work as hidden compatibility commands; the refreshed PDFs use `run`. See the current command reference for reviewed-ID selection and all mode combinations.

The script verifies the immutable source hashes and exact line selection before writing, creates a disposable index, records all command results, creates a local Git commit using the explicitly synthetic `ACORN Synthetic Demo` identity, and renders the PDF twice. It asserts that retries, rejected apply and both report runs preserve the correct index. `evidence.json` is the local report manifest; `remotePublication` is false.

Appendix A — Confirmation acceptance

The October 24 confirmation checks and interactive rehearsal accepted Yes / No behavior, cancellation, stale permissions and replay protection. The consolidated `run` API then passed ten terminal checks, the unattended rehearsal, and the interactive rehearsal. The latter used three prompts. Recorded final gates passed warning-free lint, 2,694 tests (17 skipped), book and wiki validation.

These are dated synthetic-local results, not a new run of sanitized fixtures. They sent no messages or remote updates. Current commands preserve explicit approval and exact-backup behavior; source records retain the detailed test outcomes.

Appendix A — Exact trace

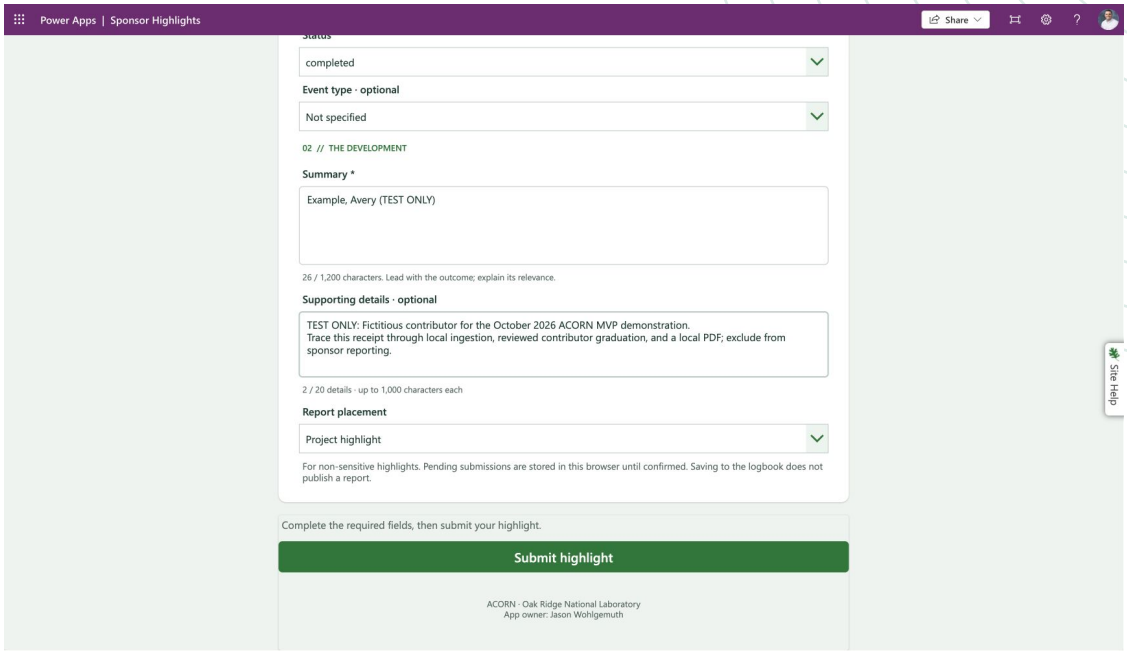
Evidence	Value
GitLab project / branch	`22464` / `main`
Inbox	`.acorn/intake/logbook.jsonl`
Submission UUID	`4313fcfc-0201-441e-8bda-c77718a31dcf`
Immutable source commit	`3090aa5714b4c6ff59c5dd0b6ecba999f918caa2`
Source parent	`7dd33b6d6094ab4d0b2ce13d8c58296a210064d5`
Append run	`08584087230445394451842398555CU19`
Exact replay run	`08584087227067729212084836766CU19`
Local entry	`form-4313-fcfc-0201-441-e-8-bda-c-77718-a-31-dcf`
Selected source	Line 6 of six lines; complete original bytes including LF
Snapshot SHA-256	`76f0e36f637b5611cca8aa6c69ce5db13be00a0b5e17b2dbfe71c20a38177384`
Selected line SHA-256	`c8496e27ce6379296d19c194305b2f17f028f2e3314230de9ce8adc7b28a2c58`
Reporting window	`2026-10-24T00:00:00Z` inclusive to `2026-10-25T00:00:00Z` exclusive

The replay run reached **Queued replay** after reading the prior immutable commit and its parent; the new-write branch was skipped. Response values remain protected in run history. The saved flow's replay response references that verified prior commit. The original app receipt and the downloaded snapshot establish the UUID/commit mapping.

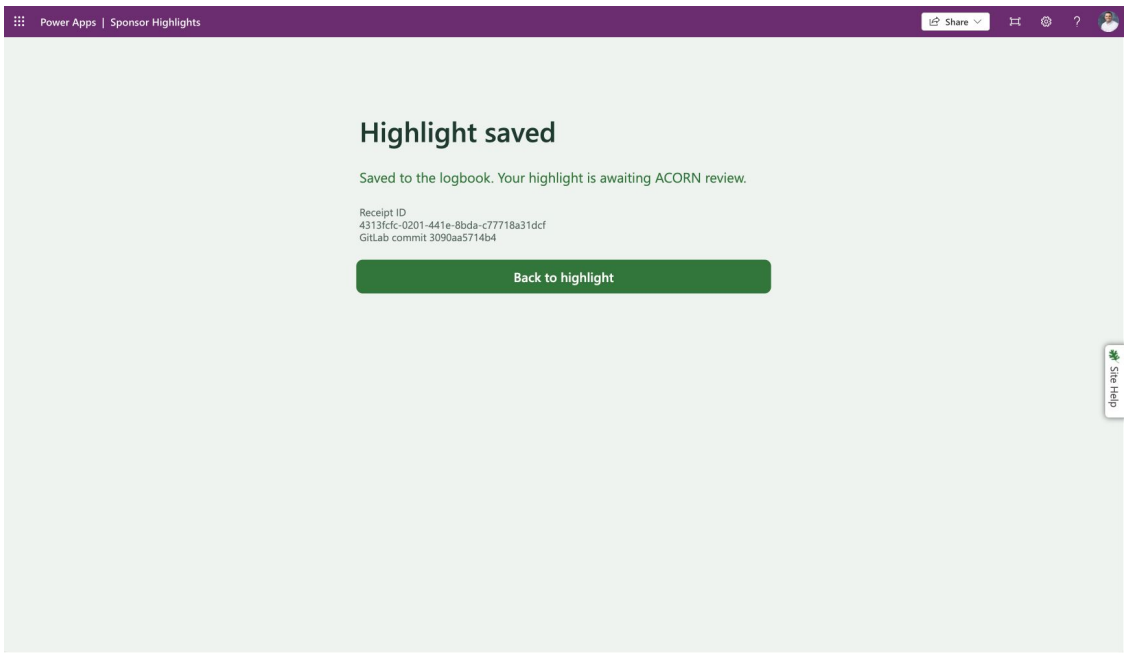
`_assets/baseline.json` is a separate synthetic local project, not a downloaded canonical project index. `_assets/selected.jsonl` is a sanitized illustration of the captured data; historical hashes apply to the original. `_assets/rejected.jsonl` intentionally changes the project selector and is a separate negative fixture, never submitted to the live flow. The index's contributor and selected archive flag are the only semantic changes between `_assets/ingested.json` and `_assets/graduated.json`.

Appendix A: captured GitLab form and receipt

Historical evidence captured October 24, 2026. UI labels or paths in these screens may differ from current commands.



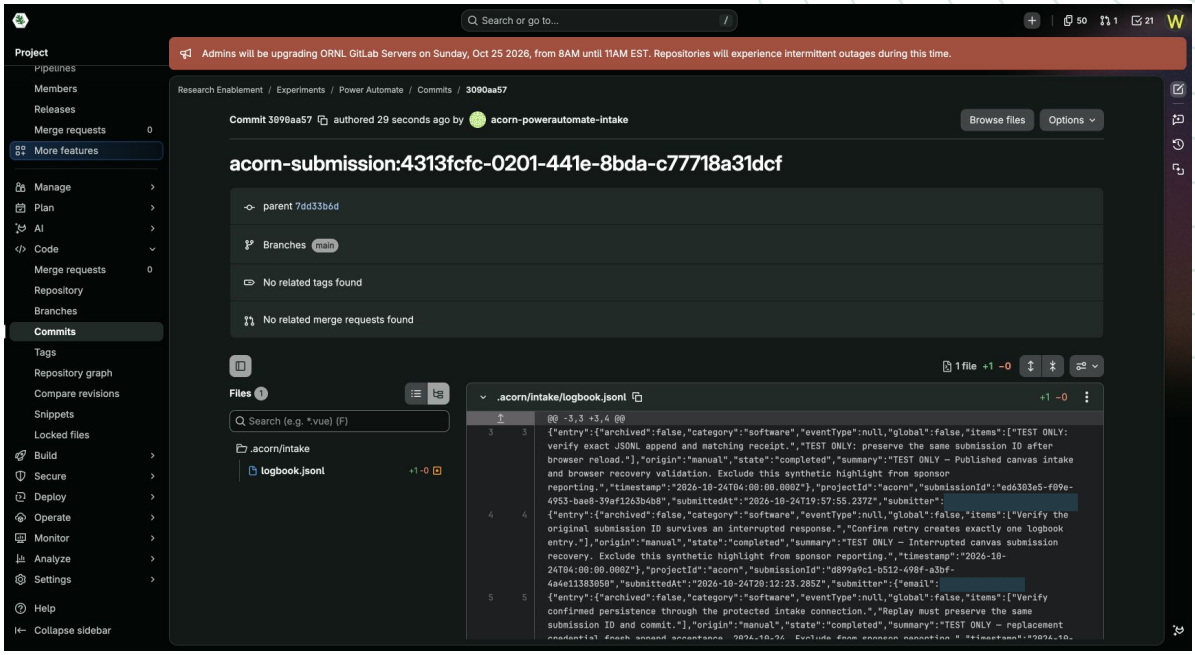
Prepared TEST ONLY personnel highlight.



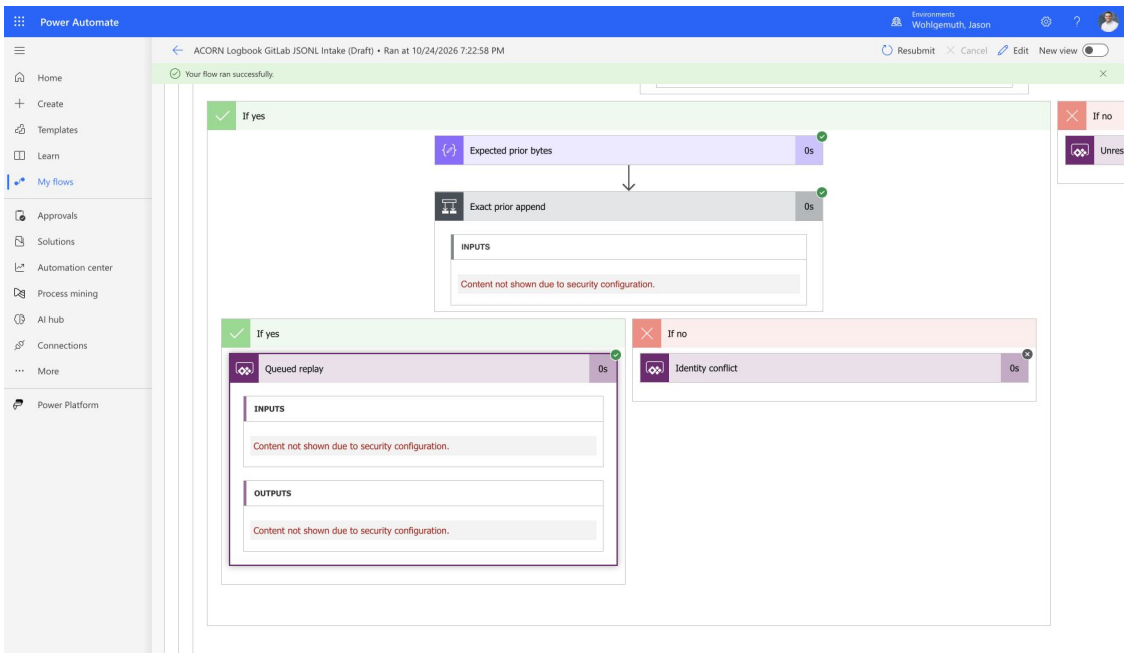
Saved receipt: capture is confirmed; content review follows.

Appendix A: captured append and replay

Historical evidence captured October 24, 2026. UI labels or paths in these screens may differ from current commands.



GitLab commit: exactly one added JSONL line.



Exact replay: the prior immutable commit is reused.

Appendix A: historical local report

Historical evidence captured October 24, 2026. UI labels or paths in these screens may differ from current commands.



Local TEST ONLY report. The retained entry is archived; this is not remote publication.